Week 9 Lab: Git & GitHub

Submission Requirements

You are required to submit **one Word document (.docx)** containing screenshots **A–F** listed in the table below, along with the three required links.

Create a new document and name it "FirstName_LastName_Week9Lab".

Screenshot	Lab Reference	What Students Must Show
A	Part 1, Step 13	VS Code showing the three files (`index.html`, `about.html`, `contact.html`) and the Git commit history.
B	Part 2, Step 9	GitHub repository showing the commits, including **Add bio to about page**.
C	Part 3, Step 5	GitHub repository showing `.gitignore`, with `.env` absent from the tracked files.
D	Part 4, Step 5	Working Netlify website showing the Home, About, and Contact navigation.
E	Part 5, Step 11	GitHub pull request showing **Merged**, with `gallery` merged into `main`.
F	Part 5, Step 16	VS Code showing `gallery.html` on local `main`, plus the terminal output of `git branch -a` after pruning.

Required Links

Include the following three clickable links in the same Word document.

Link	Description	URL
Link 1	GitHub repository URL	
Link 2	Netlify website URL	
Link 3	Merged pull request URL	

Important: Label all screenshots A–F, ensure they are clear and readable, and include your full name and student ID in the Word document.

Goal: build a small website, record changes in Git, publish and synchronize it with GitHub, protect a sample secret using .gitignore, deploy it to Netlify, and add a new page using a branch and a pull request. Finish by updating your local main branch after the merge on GitHub.

Before Class (Most Steps Already Completed)

1. Install **Git** (git-scm.com) and **VS Code** (code.visualstudio.com).
2. Create or sign in to a free **GitHub** account (github.com) and a free **Netlify** account (netlify.com). You can sign up to Netlify with your GitHub account.
3. Set up **GitHub Copilot** in VS Code: press **Ctrl+Alt+I** to open the Copilot chat panel. If nothing appears, press **Ctrl+Shift+X**, search **GitHub Copilot Chat** (publisher: GitHub) and click **Install**. Sign in with your GitHub account; the free plan is enough.
4. Type *Hello* in the Copilot chat to confirm it answers.

Do not use a OneDrive-synced folder. Use a local folder such as `C:\Users\<your-user>\Projects\project1`. OneDrive syncing can interfere with Git and damage the repository.

Part 0 · Setup check (Most Steps Already Completed)

1. **Open the terminal.** In VS Code, choose **Terminal ▸ New Terminal** (or press **Ctrl +** `).
2. **Check that Git is installed.** Run `git --version`. A version number means Git is available. If Git is "not recognized", contact the instructor.
3. **Set your Git author name.** Use the name you want attached to your commits: `git config --global user.name "Your Name"`
4. **Set your Git author email.** Use an email verified on GitHub, or your GitHub no-reply email (GitHub ▸ Settings ▸ Emails) to keep your address private: `git config --global user.email "you@example.com"`
5. **Verify and create your project folder.** Run `git config --global --list` and check your name and email. Create the **project1** folder in File Explorer, outside OneDrive.

Checkpoint: Git works, your author information is set, and your project folder is ready.

Part 1 · First repository and commits

1. **Open the project folder.** In VS Code: **File ▸ Open Folder ▸ project1**.
2. **Create index.html with Copilot (or by hand).** Open Copilot chat (**Ctrl+Alt+I**), choose **Agent** mode, and keep **Manual permissions** on so you approve each action. Send this prompt: *Create one file, index.html, under 40 lines, with a heading showing my name, [Your Name], and one sentence about me. No CSS framework or JavaScript.* If Copilot asks to run a command (for example, starting a local server), click **Skip**. Read the file, then click **Keep**. Double-click index.html in File Explorer to open it in your browser.
3. **Initialize the local Git repository.** Open **Source Control** (**Ctrl+Shift+G**) and click **Initialize Repository**. This creates the hidden **.git** folder. Never change its contents by hand. Equivalent: `git init`
4. **Make sure the branch is named main.** Check the branch name at the bottom-left of VS Code. If it is not **main** (older setups use *master*), rename it in the terminal before publishing: `git branch -M main`
5. **Stage index.html.** In Source Control, click **+** beside the file; it moves to **Staged Changes**. Equivalent: `git add index.html`

6. **Create the first commit.** Type **Add home page** in the message box and click **Commit**. Equivalent: `git commit -m "Add home page"`
7. **Make a temporary change.** Edit the h1 text in index.html and save. The file shows **M** (modified) in Source Control.
8. **Review the difference (diff).** Click index.html under **Changes**. The old line shows in red and your new line in green.
9. **Discard this uncommitted change.** Right-click index.html ▸ **Discard Changes** and confirm. The file returns to your last commit. A discarded edit cannot be recovered. Equivalent: `git restore index.html`
10. **Edit the introduction by hand and commit.** Without Copilot, add your program of study to the introduction. Save, stage and commit **Edit intro by hand**. Equivalent: `git add index.html` then `git commit -m "Edit intro by hand"`
11. **Add About and Contact pages.** Prompt Copilot: *Create about.html and contact.html, each under 40 lines, with a heading and a paragraph. Add navigation linking index.html, about.html and contact.html on all three pages.* Or write the pages by hand.
12. **Inspect and test the three pages.** Click each file in Source Control to review its diff. Open index.html in a browser and confirm the Home, About and Contact links work. Fix any broken link. Not happy with Copilot's work? Right-click ▸ **Discard Changes** and try again.
13. **Stage and commit the pages.** Stage all changes with the **+** on the **Changes** header and commit **Add about and contact pages**. Source Control should now show no changes. Equivalent: `git add index.html about.html contact.html` then `git commit -m "Add about and contact pages"`

Git tracks files, not folders: an empty folder never appears in Source Control until it contains a file.

Checkpoint: main has 3 commits, the three linked pages work, and Source Control is clean. Git records Copilot's changes and yours the same way: a commit is a commit, whoever wrote the code.

Part 2 · Publish to GitHub and synchronize

1. **Confirm you are on main.** Read the branch name at the bottom-left of VS Code before publishing.
2. **Click Publish Branch.** In Source Control, click **Publish Branch**. If asked, click **Allow**, sign in to GitHub in the browser, then click **Open Visual Studio Code**.
3. **Choose repository visibility.** Choose **public** for easy instructor access, or **private** if required.
4. **Open the repository on GitHub.** Click **Open on GitHub**. Check that the repository exists and its default branch is **main**. Your link looks like `github.com/<username>/project1`.
5. **Review the commit history.** Click **Commits** and confirm your 3 commits appear.
6. **Change about.html.** In VS Code, add a short bio paragraph to about.html and save.
7. **Stage and commit.** Commit **Add bio to about page**. This commit exists only on your computer for now; the button reads **Sync Changes 1↑**. Equivalent: `git add about.html` then `git commit -m "Add bio to about page"`
8. **Click Sync Changes.** Sync pulls any new commits from GitHub, then pushes yours. Handle any prompt or conflict (ask for help with conflicts). Equivalent: `git pull` then `git push`
9. **Verify on GitHub.** Refresh GitHub. Open the new commit to see exactly which lines changed.

A commit saves on your computer only. Nothing reaches GitHub until you sync (push).

Checkpoint: GitHub main has 4 commits and shows the new bio.

Part 3 · Keep secrets out with .gitignore

1. **Create a fake .env file.** In project1, create **.env** with this harmless line: `API_KEY=not-a-real-key`. Never use a real password or key in this lab.
2. **Confirm .env appears as untracked.** It shows under **Changes** with **U**. Do not stage or commit it.
3. **Create .gitignore.** In the project root, create a file named **.gitignore** containing one line: `.env`. (Shortcut: right-click .env in Source Control ▸ **Add to .gitignore** does the same.)
4. **Commit only .gitignore and sync.** Confirm .env has disappeared from Source Control. Commit **Add .gitignore** and click **Sync Changes**. Equivalent: `git add .gitignore`, `git commit -m "Add .gitignore"`, `git push`
5. **Verify on GitHub.** .gitignore is present; .env is not. Note: .gitignore does not stop tracking a file that was already committed. If a real secret is ever published, revoke or replace it immediately; deleting the file is not enough, because it stays in the history.

Checkpoint: .gitignore is on GitHub and the fake .env file is not.

Part 4 · Deploy the website on Netlify

1. **Start a deployment.** Sign in to Netlify and choose **Add new project** ▸ **Import an existing project** (labels may vary). Connect GitHub if asked.
2. **Select your repository.** Choose **project1**. Authorize repository access if Netlify asks.
3. **Confirm the settings.** Production branch: **main**. Build command: leave empty. Publish directory: the repository root (leave blank if blank means root). Give the site a name such as `yourname-project1`.
4. **Deploy and open the site.** Wait for a successful deploy status, then open your **netlify.app** link.
5. **Test navigation and auto-deploy.** Check all three links. Confirm the site is linked to GitHub with automatic production deploys: every push to main will now update the live site. If you see “Page not found”, check that index.html is in the publish directory.

Checkpoint: a working Netlify site shows Home, About and Contact, and main is the production branch.

Part 5 · Branch, pull request, merge and clean up

Build the feature on a branch

1. **Create a branch named gallery.** Click **main** at the bottom-left ▸ **Create new branch…** ▸ type **gallery**. The status bar now shows gallery. Equivalent: `git switch -c gallery`
2. **Add the gallery feature.** Create gallery.html (by hand or with Copilot) with a heading and three captions. Add a Gallery link to the navigation on all four pages. Test the pages locally.
3. **Commit the feature.** Stage the files and commit **Add gallery page**. Equivalent: `git add .` then `git commit -m "Add gallery page"`
4. **Commit a small improvement.** Add a subtitle to gallery.html and commit **Polish gallery page**. Equivalent: `git add gallery.html` then `git commit -m "Polish gallery page"`
5. **Publish the branch.** Click **Publish Branch**. On GitHub, confirm gallery appears in the branch menu. Equivalent: `git push -u origin gallery`

See branches in action

6. **Switch to main.** Click **gallery** in the status bar ▸ choose **main**. gallery.html disappears, because main does not contain it yet. Equivalent: `git switch main`

7. **Switch back to gallery.** gallery.html reappears. Switching branches changes the files in your folder to that branch's snapshot. Equivalent: `git switch gallery`
8. **Check the live site.** Your Netlify site still has no Gallery page: production follows main, not your unmerged branch.

Pull request and merge on GitHub

9. **Open a pull request.** Click **Compare & pull request** (or **Pull requests ▸ New**). Set base: **main**, compare: **gallery**. Title: **Add gallery page**. Add a one-sentence description and click **Create pull request**.

Note – When Does "Compare & Pull Request" Appear on GitHub?

After pushing or synchronizing a branch, GitHub may automatically display the **Compare & pull request** button when it detects a recently pushed branch with changes that can be merged.

However, this button does not always appear.

Situation	"Compare & pull request"
Newly pushed branch with changes relative to `main`	Often appears
Existing branch with newly pushed commits	May appear
Branch identical to `main`	Usually does not appear
Working directly on `main`	Does not appear for comparing `main` to itself
Branch already merged	Generally not offered for the same changes
Branch with no recent push activity	Banner may no longer appear

What should you do in this lab?

After pushing the `gallery` branch to GitHub:

1. Open your GitHub repository.
2. If **Compare & pull request** appears, click it.
3. If it does not appear, select **Pull requests → New pull request**.
4. Select **base: main** and **compare: gallery**.
5. Review the differences and create the pull request.

Remember: Pushing or synchronizing a branch does not automatically create a pull request or merge branches. You can also compare branches other than `main`.

10. **Inspect the proposed changes.** Open **Files changed** and confirm only your intended edits appear. Opening a pull request proposes a merge; it does not merge anything yet.
11. **Merge the pull request.** Choose **Create a merge commit**, then **Merge pull request ▸ Confirm merge**. If a review or protection rule blocks the merge, ask the instructor rather than bypassing it. Netlify now redeploys the site.

    After merging, copy the URL of the merged pull request from your browser. You will need this link for Submission (Link 3).

12. **Delete the gallery branch on GitHub.** Click **Delete branch** on the pull request page. This removes the branch name only; the merged changes stay in main.

Bring your computer up to date

The merge happened only on GitHub. Your local main does not know about it yet.

13. **Update your local main.** In VS Code, switch to **main**, then click **Sync Changes** (or ⋯ ▸ **Pull**). Equivalent: `git switch main` then `git pull origin main`
14. **Check gallery.html is on main.** Open it in VS Code and check the updated menus. You do not repeat the merge locally.
15. **Delete the local gallery branch.** Make sure you are on main. Press **Ctrl+Shift+P**, run **Git: Delete Branch…** and choose **gallery**. Equivalent: `git branch -d gallery`
16. **Remove stale references.** Press **Ctrl+Shift+P** and run **Git: Fetch (Prune)**. Then check that only main remains. Equivalent: `git fetch --prune` then `git branch -a`

Checkpoint: the pull request is merged, your local main contains the gallery, the gallery branch is gone from GitHub and your computer, and the live site has a Gallery page.